INTEGRATION LAYER V2.0

Architecture

WWMG_Source · MyCareHub · PostgreSQL | BigQuery · ConsumerApplication

Version 2.0 | 28 September 2026

Contents: Architecture, Implementation Plan

Contents

Integration Layer V2.0: Architecture 3

1. Purpose . . . 3
2. System view . . . 3
3. Choosing PostgreSQL or BigQuery per workflow . . . 4
4. Request lifecycle (both backends) . . . 4
5. End-to-end flows . . . 4
6. Components . . . 5
7. Data model . . . 6
8. Data warehouse . . . 6
9. Security . . . 6
10. Deployment . . . 6

Integration Layer V2.0: Implementation Status and Roadmap 7

Requirements for V2 and where they are met . . . 7
Verification . . . 7
Design decisions . . . 7
Roadmap . . . 8

Integration Layer V2.0: Architecture

1. Purpose

Two source applications, WWMG_Source and MyCareHub, send JSON requests to an Integration Layer (IL). The IL validates each request against its workflow's JSON Schema and stores it in the workflow's storage backend: PostgreSQL or Google BigQuery, chosen per workflow. ConsumerApplication picks requests up from both backends, processes them and writes the result back. The IL then returns the result to the source application by polling or by signed callback. A data warehouse combines both backends for analytics, and an operations console runs, observes and controls everything.

What changed from V1

Area	V1	V2
Producer	App1 (mock)	WWMG_Source (orders) and MyCareHub (member-facing care portal)
Consumer	App2	ConsumerApplication, one queue worker per backend
Storage	PostgreSQL only	PostgreSQL or BigQuery per workflow (`workflow_registry.storage_backend`)
Front-facing app	none	MyCareHub portal: members submit care requests and get notified by callback
Warehouse	PostgreSQL requests	PostgreSQL and BigQuery requests, with a `storage_backend` dimension
Console	black and red theme	black and red or light theme (toggle); backend-aware in every view
Docs	architecture and plan	+ step-by-step runbook, API reference, OpenAPI files

2. System view

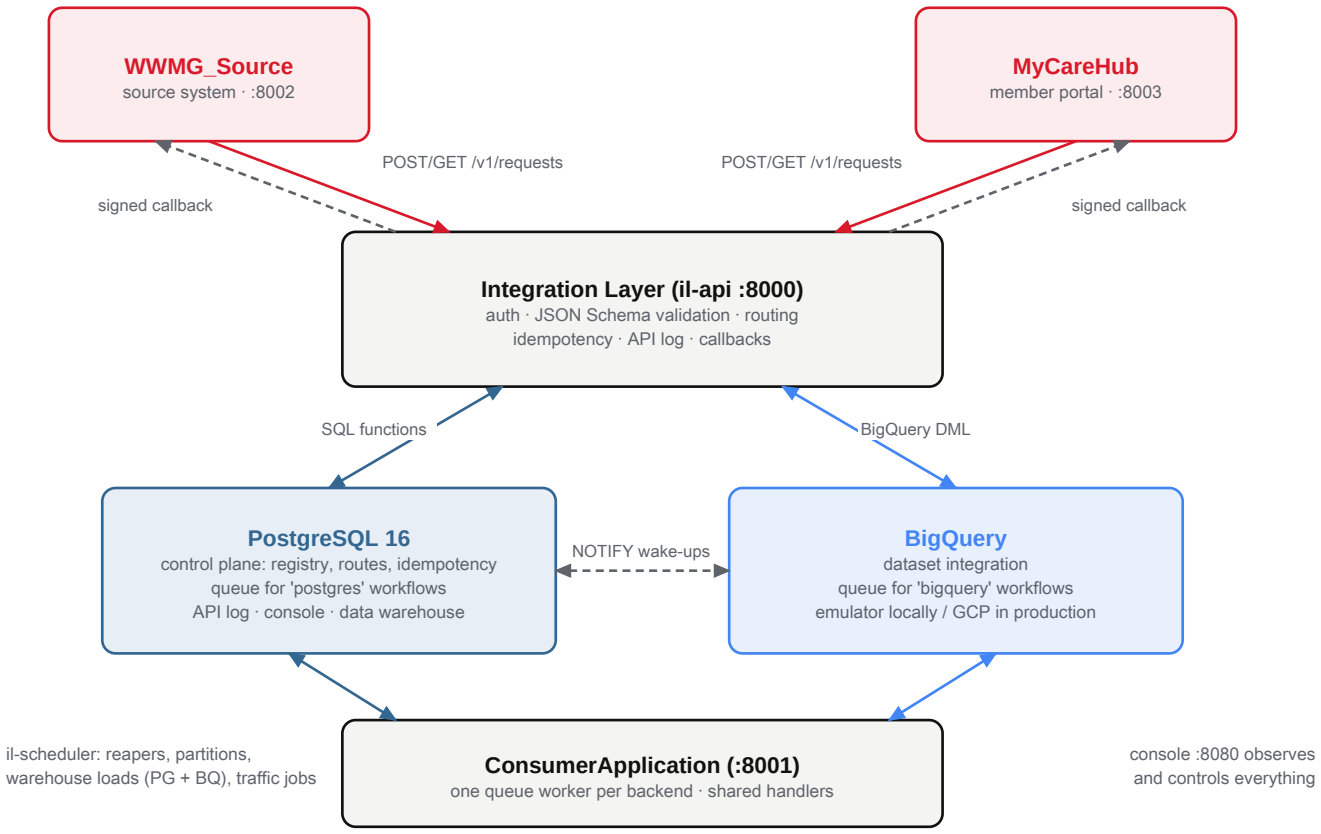


Figure: V2 system architecture

PostgreSQL is always the control plane. It holds the workflow registry, request routing, idempotency keys, the API call log, the console's jobs and payload library, and the data warehouse. It is also the coordination bus: `NOTIFY`

`request_new` and `NOTIFY request_done` wake consumers and the callback dispatcher for both backends. BigQuery is a second data plane: it stores and queues requests, and records the status history, for workflows set to `bigquery`.

3. Choosing PostgreSQL or BigQuery per workflow

`integration.workflow_registry.storage_backend` is `postgres` or `bigquery`. It is edited in the console (Workflows tab) or through `PUT /v1/admin/workflows/{name}`. A change applies to new requests; existing requests stay where they were stored and still complete there.

Queue claim	`FOR UPDATE SKIP LOCKED`	one `UPDATE … WHERE request_id IN (SELECT … LIMIT n)` stamped with a claim token; BigQuery serialises conflicting DML, so claims stay exclusive
Wake-up	`NOTIFY request_new` / `request_done`	the same NOTIFY channels (sent by the IL and ConsumerApplication), plus polling
Typical latency	milliseconds	about 1–3 s per statement (emulator and real BigQuery)
Cost	fixed (your server)	per query; poll intervals are configurable (`CONSUMER_BQ_POLL_SECONDS`)
Best for	interactive, low-latency flows	analytics-heavy data or data already in GCP; slower SLAs are fine
Seeded workflow	`order_validation` (WWMG_Source)	`care_request` (MyCareHub)

Idempotency and routing for BigQuery. BigQuery has no unique constraints, so the IL first calls `integration.register_bq_request()` in PostgreSQL. That inserts the idempotency key (unique per source app) and a row in `integration.request_routes`, atomically. Only then does it insert into BigQuery. If the BigQuery insert fails, `unregister_bq_request()` undoes the registration and the caller gets 503 BACKEND_UNAVAILABLE. Look-ups by id check PostgreSQL first, then `request_routes`, so a request is always found and its ownership (source app) is enforced for both backends.

Portability of the BigQuery code. The same SQL runs on real BigQuery and on the local emulator: the clock is always passed as a `@now` parameter, NULL parameters become typed NULL literals, and rows changed by an UPDATE are counted through an `op_token` column. JSON is stored as STRING.

4. Request lifecycle (both backends)



Retries back off 30 s × 2^n (maximum 1 h). A worker's lease is extended by heartbeats; the reapers (`reap_expired_leases`, `reap_expired_leases_bigquery`) recover requests whose worker died. Every transition is written to `request_event_log` in the backend that holds the request.

5. End-to-end flows

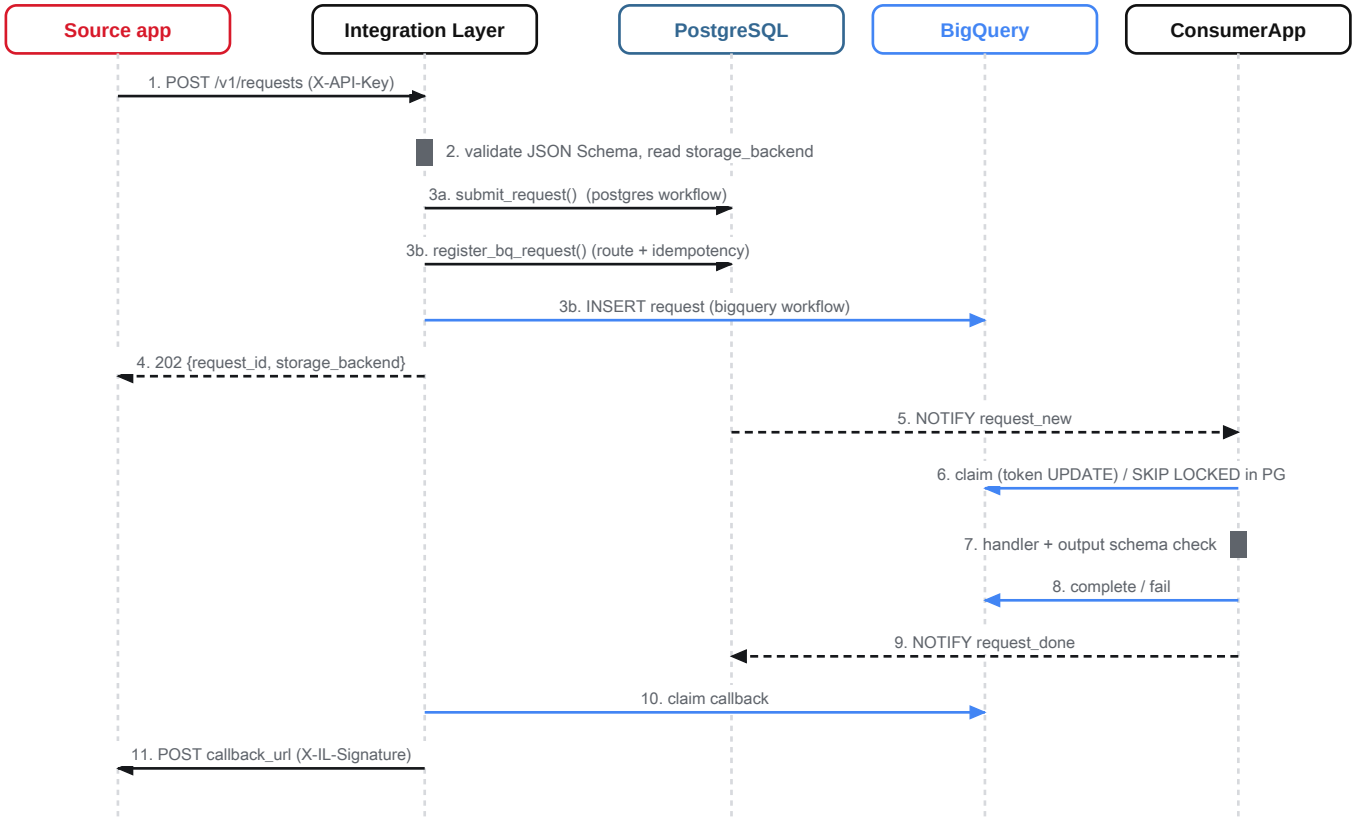


Figure: request flow for both backends (3a = PostgreSQL workflow, 3b = BigQuery workflow)

MyCareHub example: a member asks for an appointment → `care_request` (BigQuery) → ConsumerApplication books a slot (`SCHEDULED`, `scheduled_for`) → callback → MyCareHub shows a notification and the updated request card.

6. Components

Component	Code	Responsibility
Integration Layer	`integration_layer/`	REST API, validation, routing to PG/BQ, idempotency, API call log, callback dispatcher (both backends), admin workflow registry
Shared library	`common/ilcommon/`	`bigquery_store.py` (BigQuery data plane), `il_client.py` (source-app client + signature check), `templating.py` (payload placeholders)
ConsumerApplication	`consumer_application/`	Workers for `postgres` and `bigquery` queues; handlers `order_validation`, `care_request`; output validation; pause/resume per backend
WWMG_Source	`wwmg_source/`	Source system: submits orders, receives signed callbacks
MyCareHub	`mycarehub/`	Member-facing portal (light/dark): care request form, live request status, notifications from callbacks
il-scheduler	`integration_layer/scheduler.py`	System jobs (reapers, partitions, warehouse loads for PG and BQ) and console traffic jobs, in separate threads
migrate	`integration_layer/migrate.py`	Applies `db/*.sql`, creates login users, creates the BigQuery dataset and tables
Console	`console/`	Dashboard, Analytics, Jobs, Payloads & send, Requests, Telemetry, Workflows; black and red / light theme toggle

7. Data model

PostgreSQL (migrations `db/001–004`):

- `integration.workflow_registry`: workflow, target app, input/output JSON Schemas, `schema_version`, `max_retries`, **`storage_backend`**
- `integration.request_metadata`: every PostgreSQL-stored request, with all metadata (input/output JSON, API URL and method, server details, timestamps, status, `success_flag`, workflow status, retry/lease and callback columns); partitioned by month
- `integration.request_routes`: id → backend and source app, for BigQuery-stored requests
- `integration.idempotency_keys` (with `storage_backend`), `integration.api_call_log`, `integration.request_event_log`
- `dwh.*`: star schema; `fact_request.storage_backend` distinguishes the origin
- `console.*`: payload samples, traffic jobs (with `source_app`), job runs and submissions, system jobs

BigQuery (dataset `integration`, created by `migrate`):

- `request_metadata`: the same columns as PostgreSQL (JSON as STRING) plus `claim_token`, `callback_claim_token` and `op_token`; partitioned by `request_date` and clustered on real BigQuery
- `request_event_log`: status history

8. Data warehouse

`dwh.merge_changes(p_backend, p_job)` merges a staged batch (`_chg`) into the star schema. The `dwh_load_incremental` system job stages changed PostgreSQL rows; `dwh_load_bigquery` reads BigQuery rows changed since its watermark (`fact_request_bigquery`) and stages them. Both run every 60 s by default and can be triggered from the console. Analytics can filter by backend and compares the two in the "By storage backend" table.

9. Security

- API keys per source app (`IL_API_KEYS=WWMG_Source:…,MyCareHub:…`); apps see only their own requests
- Callbacks signed with HMAC-SHA256 (timestamped; 5-minute replay window); host allow-list against SSRF
- Least-privilege PostgreSQL roles: `il_service`, `consumer_worker` (queue functions only), `il_scheduler`, `dwh_etl`, `dwh_reader`, `ops_viewer` + `console_app` (console)
- BigQuery: emulator locally; on GCP use a service account limited to the dataset (BigQuery Data Editor + Job User) mounted read-only (`docker-compose.gcp.yml`)
- The console holds the admin key and is published on 127.0.0.1 only; MyCareHub uses synthetic demo members

10. Deployment

`docker compose up -d --build` starts 9 containers: `postgres`, `bigquery` (emulator), `migrate` (one-shot), `il-api`, `il-scheduler`, `consumer-application`, `wwmg-source`, `mycarehub`, `console`. `docker-compose.gcp.yml` switches every service to real BigQuery. Step-by-step commands are in RUNBOOK.md; every endpoint is in API_REFERENCE.md.

Integration Layer V2.0: Implementation Status and Roadmap

Requirements for V2 and where they are met

#	Requirement	Delivered
1	Updated documents and code in a new folder `V2.0`	This repository; V1 is unchanged in its own repository
2	Keep PostgreSQL, add BigQuery; each workflow chooses its source	`workflow_registry.storage_backend`; `ilcommon/bigquery_store.py`; routing in the IL; per-backend workers; console editor
3	Rename app1 to WWMG_Source	`wwmg_source/` service, API key and source_app `WWMG_Source`
4	Rename app2 to ConsumerApplication	`consumer_application/`, role `consumer_worker`, target_app `ConsumerApplication`
5	Connect a new front-facing application MyCareHub	`mycarehub/` portal: own API key, `care_request` workflow, signed callbacks, member notifications
6	Document with step-by-step launch commands and API documentation	RUNBOOK.md, API_REFERENCE.md, `docs/openapi/*.json`, PDF versions
7	Toggle between the black and red theme and a light theme	Console top-right toggle (remembered per browser); MyCareHub has the same toggle
8	Rebuild everything and push to a new repository IntLayerV2.0	This repository

Verification

Check	Result
Automated tests (PostgreSQL 16 + BigQuery emulator)	46 passed: database functions, IL API, BigQuery store (lifecycle, retries, exclusive concurrent claims, reaper, callbacks), IL routing to BigQuery, BigQuery worker, BigQuery → warehouse, jobs, templating, care_request rules
End-to-end smoke test (`scripts/smoke_test.py`)	All checks passed with every service running (both sources, both backends, callbacks, retries, idempotency, live backend switch, console)
UI review	Console (dark and light), MyCareHub (light, dark, mobile) rendered in Chrome with no console errors

Design decisions

- **PostgreSQL stays the control plane** (registry, routing, idempotency, API log, console, warehouse, NOTIFY). BigQuery is an additional data plane chosen per workflow. This gives BigQuery workflows the same guarantees (idempotency, ownership, callbacks, audit) without distributed transactions.
- **BigQuery as a queue** uses token-stamped UPDATEs. BigQuery serialises conflicting DML on a table, so claims are exclusive; it suits the stated volume (500–1,000 requests/day) but not high-throughput queues.
- **Local BigQuery** runs on the open-source emulator; `docker-compose.gcp.yml` switches to real BigQuery.

Roadmap

Priority	Item
High	Run the stack on real BigQuery in a GCP project and tune poll intervals for cost
High	TLS termination in front of il-api and MyCareHub; authentication for the console and MyCareHub (SSO)
Medium	Secrets from a vault (instead of `.env`); rotate API keys
Medium	Metrics endpoint (Prometheus) and alerts: oldest PENDING, failure rate, callback failures, BigQuery latency
Medium	Backups: PostgreSQL PITR; BigQuery table snapshots
Low	Move the warehouse to BigQuery (and load PostgreSQL requests into it) if analytics grows
Low	CI pipeline running the test suite with postgres and emulator service containers